For the Sub-image matrix F(x,y) giving the connectivity $V = \{26, 31, 58, 56, 87, 89\}$determines the following parameters:

2	3	1	2	0	87	1	1
1	87	20	0	17	89	0	2
1	26	31	31	40	56	0	0
0	31	58	56	26	58	2	0
0	58	1	1	1	0	1	1

1. Using 4-adjacency and the connectivity set V, perform connected component labeling on the image matrix. Present the resulting labeled image matrix where each connected region is assigned a unique label, and background pixels (values not in V) are assigned label 0.
2. The Chessboard distance between two pixels p and q with coordinates f(2,0) and f(0,2)
3. The City Block distance between two pixels p and q with coordinates f(1,2) and f(3,3)
4. The Euclidean distance between two pixels p and q with coordinates f(3,2) and f(2,5)
5. D_m distance between two pixels p and q with coordinates f(2,0) and f(0,2)
6. D_m distance between two pixels p and q with coordinates f(1,2) and f(3,3)
7. D_m distance between two pixels p and q with coordinates f(1,1) and f(2,3)

Answer

1. Using 4-adjacency and the connectivity set $V = \{26,31,58,56,87,89\}$

0	0	0	0	0	1	0	0
0	2	0	0	0	1	0	0
0	2	2	2	0	1	0	0
0	2	2	2	2	1	0	0
0	2	0	0	0	0	0	0

L1=L2

L1 = {F(0,5),F(1,1),F(1,5), F(2,1), F(2,2), F(2,3), F(2,5), F(3,1), F(3,2), F(3,3), F(3,4), F(3,5), F(4,1) }

2. Chessboard (D_8) between p(x, y) & q(s, t) is $D_8(p,q) = \max(|x-s|, |y-t|)$=max(|2-0|,|0-2|)=2
3. City Block distance (D_4) between p(x, y) & q(s, t) is $D_4(p,q) = |x-s| + |y-t|$=|1-3|+|2-3|=3
4. The Euclidean distance between p(x, y)&q(s, t) is $D_e(p,q) = \sqrt{(x-s)^2 + (y-t)^2} = \sqrt{(3-2)^2 + (2-5)^2} = 3.16$
5. Not possible
6. Not possible
7. **m-adjacency neighbors** of $f(1,\ 1)$ $N4 \in V$ ={$f(2,\ 1)$}

   $N_D(1,1) \in V$ ={$f(2,2)$} ***check on connectivity***

   $f(1,1)$ $N4 \in V$ ={$f(2,\ 1)$}

   $f(2,2)$ $N4 \in V$ ={$f(2,\ 1), f(3,\ 2), f(2,\ 3)$}

   $N_4(1,1) \cap N_4(2,2)$={$f(2,\ 1)$} $so\ it's\ \ not\ m\ adjacent$

   m-adjacency neighbors of $f(2,\ 1)$ $N4 \in V$={$f(2,2)$}

$N_D(2,1) \in V = f(3,\ 2)$ ***check on connectivity***

$f(2,1)\ N4 \in V = \{f(2,\ 2)\}$

$f(3,2)\ N4 \in V = \{f(2,\ 2), f(3,\ 1), f(3,\ 3)\}$

$N_4(2,1) \cap N_4(3,2) = \{f(2,\ 2)\}\ so\ it's\ not\ m\ adjacent$

m-adjacency neighbors of $f(2,\ 2)\ N4 \in V = \{f(2,3)\}$

The length of the shortest m-path=$\{f(1,\ 1)\}, f(2,\ 1), f(2,\ 2), f(2,3)\}$

D_m=n-1=4-1=3

Define the following Term:

Geometric transformation: change the spatial arrangement of pixels to correct geometric distortions that occur with non-ideal camera angles. It can be categorized into Affine transformations (Maintains parallelism) and non-affine transformations (may warp or bend the image).

Affine transformations

- Sets of parallel lines remain parallel after an affine transformation.
- It's linear transformation and can be represented **by a matrix multiplication**

- Translation



- Rotation



- scaling



- Skewing



- reflecting





```cpp
#include<opencv2/opencv.hpp>
#include"iostream"
    using namespace std;
    using namespace cv;
    int main()
    {
        ////////////////////image Translation//////////////////////
        Mat src_t, dst_t;
        src_t = imread("t.jpg", 0);
        namedWindow("orginal_t", 0);
        imshow("orginal_t", src_t);
        waitKey(0);
        int tx = 50, ty = 50;
        Mat TM = (Mat_<float>(2, 3) << 1, 0, tx, 0, 1, ty);
        warpAffine(src_t, dst_t, TM, src_t.size());
        namedWindow("Translation", 0);
        imshow("Translation", dst_t);
        imwrite("translation.jpg", dst_t);
        waitKey(0);
```

```
    ////////////////////////// Image Rotation//////////////////////////
    Mat src_r, dst_r;
    src_r = imread("r.jpg", 0);
    namedWindow("orginal_r", 0);
    imshow("orginal_r", src_r);
    waitKey(0);

    //resize(src_r, src_r, Size(src_r.cols, src_r.cols));
    Mat R = getRotationMatrix2D(Point2f(src_r.cols / 2, src_r.rows / 2), 90, 1);
    warpAffine(src_r, dst_r, R, src_r.size());
    namedWindow("rotation", 0);
    imshow("rotation", dst_r);
    waitKey(0);
    imwrite("rotation.jpg", dst_r);
////////////////////////////Image Skewing//////////////////
Mat src_s, dst_s;
src_s = imread("s3.jpg", 0);
namedWindow("orginal_s", 0);
imshow("orginal_s", src_s);
waitKey(0);
Point2f src_p[3];
src_p[0] = Point2f(0, 0);
src_p[1] = Point2f(src_s.cols - 1, 0);
src_p[2] = Point2f(100, src_s.rows - 1);
Point2f dst_p[3];
dst_p[0] = Point2f(0, 0);
dst_p[1] = Point2f(src_s.cols - 1, 0);
dst_p[2] = Point2f(0, src_s.rows - 1);
Mat SM = getAffineTransform(src_p, dst_p);
warpAffine(src_s, dst_s, SM, src_s.size());
namedWindow("skewing", 0);
imshow("skewing", dst_s);
imwrite("skewing.jpg", dst_s);
waitKey(0);
```

```
////////////////////////////////////Image Zoom/////////////////
Mat src, dst_z;
src = imread("z.jpg", 0);
namedWindow("Orginal_z", 0);
imshow("Orginal_z", src);
waitKey(0);
Mat src_zoom = src(Rect(0, 0, src.cols / 2, src.rows / 2));
resize(src_zoom, dst_z, Size(), 2, 2, 0);
namedWindow("image_zoom", 0);
imshow("image_zoom", dst_z);
waitKey(0);
imwrite("zoom.jpg", dst_z);
////////////////////////image flip///////////////
Mat src_f, dst_f;
src_f = imread("f.jpg", 0);
namedWindow("orginal_f", 0);
imshow("orginal_f", src_f);
waitKey(0);
flip(src_f, dst_f, 0);
namedWindow("flip around y", 0);
imshow("flip around y", dst_f);
imwrite("flip around y.jpg", dst_f);
waitKey(0);
flip(src_f, dst_f, 0);
namedWindow("flip around x", 0);
imshow("flip around x", dst_f);
waitKey(0);
imwrite("flip around x.jpg", dst_f);
waitKey(0);
flip(src_f, dst_f, -1);
namedWindow("flip around xy", 0);
imshow("flip around xy", dst_f);
imwrite("flip around xy.jpg", dst_f);
waitKey(0);
 }
```